

PAYSYNC

Verified Bulk Mobile Money Payment Gateway

System Development Lifecycle Document

Prepared by: Omule Brian Fredrick

***Bachelor Of Science in Computer Science Engineering |
Diploma Telecommunications Engineering
Certificate In ICT***

Organization: Digital Jetplus

Document Status: Living Document — reflects both delivered work and planned roadmap

15 August 2026

Table of Contents

1. Executive Summary.....	3
2. System Overview.....	3
3. Technology Stack.....	5
4. System Architecture.....	7
5. Database Design.....	8
6. Security Model.....	9
7. Development Methodology.....	10
8. Phased Development Plan.....	11
Phase 1 — Discovery, Requirements & System Design.....	11
Phase 2 — Database & Core Data Pipeline.....	11
Phase 3 — Payment Engine & Adapter Architecture.....	12
Phase 4 — Authentication, Authorization & Security Hardening.....	12
Phase 5 — Frontend Application Development.....	13
Phase 6 — Live Payment Gateway Integration & Sandbox Verification.....	14
Phase 7 — Quality Assurance & Testing.....	14
Phase 8 — Deployment & Infrastructure.....	15
Phase 9 — Monitoring, Support & Maintenance.....	15
Phase 10 — Post-Launch Enhancement Roadmap.....	16
9. Current Project Status Summary.....	17

1. Executive Summary

PaySync is a verified bulk mobile money disbursement platform purpose-built for the Ugandan market, enabling organizations to pay large numbers of recipients — staff, agents, vendors, contractors — through MTN Mobile Money and Airtel Money, with mandatory identity verification performed before any payment is sent. This document is the full system development lifecycle record for PaySync: what problem it solves, the complete technology stack behind it, the system architecture, and a phase-by-phase account of what has been built, what is in progress, and what remains, with every module named explicitly rather than deferred as a future assumption.

As of this writing, Phases 1 through 5 — covering system design, the core data pipeline, the payment engine, authentication, and the full frontend application — are functionally complete and have been verified against a live running instance of the system, including a real PostgreSQL database and a real Express API server. Phase 6 onward requires external actions outside engineering control — registering with MTN and Airtel's developer programs, provisioning a production Firebase project, and selecting hosting infrastructure — and is documented in full detail below rather than left as an open question.

2. System Overview

2.1 Problem Statement

Organizations across Uganda routinely disburse money to large groups of people — salaries, field-agent commissions, vendor payments, stipends — and increasingly do so through bank transfer or mobile money. A recurring, costly failure mode in this process is that many banking and payment interfaces do not display the registered name behind an account or phone number before a transfer is confirmed. A single mistyped digit, a stale contact list, or a recycled phone number can silently redirect real money to the wrong person, and the mistake is often only discovered after the funds have already moved and are difficult or impossible to recover.

At the same time, no existing tool in the Ugandan market combines bulk mobile money disbursement across both major networks (MTN and Airtel) with mandatory, pre-payment name verification and a deliberate, human-in-the-loop batching workflow that prevents an operator from paying an entire list unattended.

2.2 Proposed Solution

PaySync addresses this directly. An operator uploads a contact list (spreadsheet or manual entry), and the system queries MTN's and Airtel's own APIs to retrieve the actual registered name behind each phone number — the same verification banking apps perform — before a single payment is sent. Contacts are processed in confirmed batches of a configurable size (ten, by default), with an explicit human confirmation required before each batch is paid, and a complete, queryable audit trail records every action taken by every user.

2.3 System Name & Branding

The system is named PaySync. The name was chosen to communicate its core function in two words: keeping a payment list and the real-world identity of each recipient in sync before money moves. The visual identity — a dark “vault” palette of deep bottle-green with a brass accent, and a hand-stamped verification seal as the signature mark — reinforces the same idea: this is a system whose entire purpose is certifying who is being paid, not merely moving funds quickly.

2.4 Core Capabilities

- Bulk contact import from CSV or Excel spreadsheets, with tolerant column-header matching.
- Manual contact capture form for smaller, ad-hoc payment runs — feeding the identical validation pipeline as a spreadsheet upload.
- Automatic Ugandan mobile network detection (MTN vs. Airtel) from phone number prefixes.
- Live name-lookup verification against MTN MoMo and Airtel Money before any contact becomes payable.
- Mandatory human review and explicit approval or rejection of any name mismatch before that contact can be paid.
- Confirmed batch payment execution — contacts are processed in fixed-size groups with an explicit confirmation prompt before each batch, never a single unattended bulk transfer.
- Atomic, race-condition-safe batch selection, guaranteeing two operators can never be assigned the same contact.
- Automatic payment reconciliation against MTN/Airtel's asynchronous transaction status, with a safe, human-gated retry path for failures.
- Full role-based access control (administrator and cashier roles), with self-service account creation and admin-gated payment permissions.
- A complete, queryable audit log of every upload, verification, approval, payment, and administrative action.

2.5 Target Users

- Cashiers / Payment Operators — upload or capture contact lists, review verification results, and execute confirmed batch payments.
- Administrators — manage user accounts and role assignments, and retain full cashier capabilities.
- Auditors / Finance Reviewers — consume the audit trail and payment ledger to confirm every disbursement is accounted for.

The initial deployment context is Asinga Technologies, a Ugandan digital marketing and technology solutions company, for internal and client-facing bulk disbursement needs.

3. Technology Stack

Every technology below is already installed, configured, and running in the current build — none are speculative. Versions shown are the exact versions in use as of this document's date.

3.1 Frontend Layer

Technology	Version	Role in the System
React	19.2.8	Component-based UI library; the foundation of the entire client application.
TypeScript	6.0.2	Static typing across the whole frontend; API response shapes are typed to match the backend exactly, catching integration bugs at compile time.
Vite	8.2.0	Build tool and development server; provides the dev proxy that forwards API calls to the backend.
React Router	7.18.2	Client-side routing across Login, Signup, Dashboard, Upload, Capture, Verify, and Batch Payment screens.
Tailwind CSS	4.3.3	Utility-first styling engine; powers the custom dark “vault” design system via CSS-based theme tokens.
Firebase JS SDK	12.17.1	Client-side authentication — sign-up, sign-in, session state, ID token retrieval.

3.2 Backend / API Layer

Technology	Version	Role in the System
Node.js	22.x (LTS)	JavaScript runtime executing the Express API server.
Express	5.2.1	REST API framework; all 15+ endpoints are built on this.
Multer	2.2.0	Multipart form-data handling for spreadsheet file uploads.
Papa Parse	5.5.4	CSV parsing for uploaded contact lists.
SheetJS (xlsx)	0.18.5	Excel (.xlsx / .xls) parsing for uploaded contact lists.
Axios	1.19.0	HTTP client used by the MTN and Airtel payment adapters to call external network APIs.
node-cron	4.6.0	In-process scheduler driving the automatic payment-reconciliation job.

3.3 Database Layer

Technology	Version	Role in the System
PostgreSQL	16.14	Primary relational data store — sessions, contacts, users, and the full audit log.
node-postgres (pg)	8.22.0	PostgreSQL driver; all queries use parameterized statements and connection pooling.

3.4 Authentication & Identity

Technology	Version	Role in the System
Firebase Authentication	—	Identity provider for both sign-up and sign-in; owns password storage and verification entirely.
firebase-admin	14.2.0	Server-side verification of Firebase ID tokens on every authenticated API request.

3.5 Payment Gateway Integrations

Technology	Version	Role in the System
MTN MoMo API	Disbursements product	Name-lookup and disbursement (payment) calls for MTN Mobile Money recipients.
Airtel Money Open API	Standard / Disbursements	Name-lookup and disbursement calls for Airtel Money recipients.

3.6 Security & Hardening

Technology	Version	Role in the System
Helmet	8.3.0	Standard HTTP security headers on every response.
CORS	2.8.6	Explicit cross-origin policy for browser access to the API.
express-rate-limit	8.6.1	Rate limiting applied API-wide, defending every endpoint against abuse.
Morgan	1.11.0	Structured HTTP request logging.
dotenv	17.4.2	Environment variable loading; paired with a custom startup validator.

3.7 Development & Build Tooling

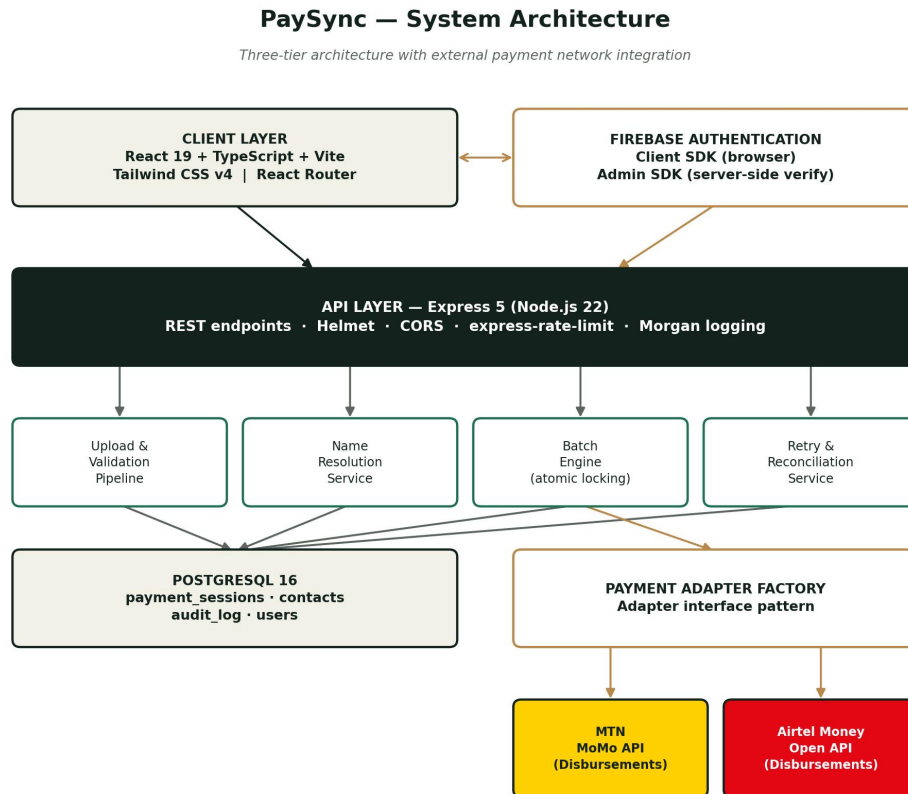
Technology	Version	Role in the System
TypeScript Compiler	6.0.2	Full-project type-checking (tsc -b) run before every production build.
eslint	1.75.0	Fast linting for the frontend codebase.
npm	bundled with Node 22	Package management for both backend and frontend.

3.8 Infrastructure & Deployment (Planned)

The following are specified for Phase 8 and are described there in full rather than left open: a managed PostgreSQL instance, a Node.js application host, static hosting for the built frontend bundle, a CI/CD pipeline, and a production Firebase project distinct from the development project used during the build phase.

4. System Architecture

PaySync follows a three-tier architecture: a React client, a Node.js/Express API layer, and a PostgreSQL data layer, with two categories of external integration sitting alongside the API — Firebase for identity, and the MTN/Airtel payment networks reached through a dedicated adapter layer.



4.1 Layer Responsibilities

- Client Layer — renders every screen, manages Firebase sign-in state, and calls the API exclusively over authenticated HTTPS requests. Holds no payment logic itself.
- API Layer — the single authority for every state change. Verifies identity on every request, enforces role permissions, and is the only layer with database and payment-network credentials.
- Services Layer — four purpose-built services (upload/validation, name resolution, batch engine, retry/reconciliation) that the API layer's routes delegate to. Each owns one part of the payment lifecycle.
- Payment Adapter Factory — an interface-based abstraction so the batch engine never needs to know whether a contact is on MTN or Airtel; adding a third network in future requires one new adapter file, not a rewrite.
- Database Layer — the single source of truth for every session, contact, user, and audit entry. All writes that must be atomic (e.g. locking a batch) use PostgreSQL row-level locking rather than application-level coordination.

4.2 Design Principles Applied

- Atomicity over convenience — batch selection uses `SELECT ... FOR UPDATE SKIP LOCKED` so two operators can never be handed the same contact, rather than relying on application-level checks that could race.

- Verification is structurally enforced, not merely suggested — the database query that selects payable contacts requires `names_match = TRUE`; a contact cannot enter a batch by any code path that skips verification.
- Adapters, not conditionals — network-specific logic (MTN vs. Airtel) is isolated behind one shared interface, keeping the batch engine and retry logic network-agnostic.
- Fail loud, fail early — missing configuration (database credentials, Firebase keys) is detected and reported at server startup, not discovered later as a cryptic runtime error.

5. Database Design

The schema consists of four core tables, evolved across three migrations as authentication requirements changed.

5.1 Core Entities

Table	Purpose
payment_sessions	One row per uploaded file or manual capture batch — tracks total contacts, batch size, and overall status (active / completed / cancelled / paused).
contacts	One row per recipient: name on file, phone number, detected network, registered name from the network, match status, amount, and payment status through its full lifecycle.
users	One row per person who has authenticated via Firebase at least once — links a Firebase UID to an application role (cashier or admin).
audit_log	Append-only history of every state-changing action: uploads, name resolutions, batch locks, confirmations, payments, approvals, skips, retries, and role changes.

5.2 Contact Payment State Machine

Every contact moves through a strict, one-directional state machine — this is what makes duplicate or premature payment structurally impossible rather than merely unlikely.

State	Meaning
pending	Loaded and validated. Eligible for batch selection only once <code>names_match = TRUE</code> .
locked	Selected into a batch, awaiting operator confirmation. Row-locked so no other batch can claim it.
processing	Operator confirmed; the payment network call is in flight.
paid	Network confirmed success. Terminal state.
failed	Network reported failure, or a status check timed out. Eligible for a bounded number of retries.
skipped	An operator explicitly removed this contact from the payable pool without paying it. Terminal state.

5.3 Migration History

- 001_init_schema.sql — original schema: payment_sessions, contacts, audit_log, and all enumerated status types.
- 002_add_users.sql — introduced the users table and linked payment_sessions / audit_log to authenticated user records.
- 003_firebase_auth.sql — migrated users from a self-managed password_hash column to a firebase_uid column; relaxed audit_log.session_id to nullable to permit account-level audit entries (e.g. role changes) that are not tied to any single payment session.

6. Security Model

6.1 Authentication

- Every account is created through Firebase Authentication — the system never stores or handles a password directly.
- Every API request (other than none — there is no public endpoint) presents a Firebase ID token, verified server-side on every call via firebase-admin.
- A new Firebase-authenticated identity is auto-provisioned a local application profile on first contact with the API, defaulting to the cashier role.

6.2 Authorization

- Two roles: cashier (upload, verify, batch-pay) and admin (all cashier capabilities, plus user and role management).
- Self-service account creation can never itself grant payment permission — every new account starts as cashier-level access under the current design, with role changes requiring an existing admin's explicit action, fully audited.

6.3 Session Management

- Firebase ID tokens are short-lived (approximately one hour) and refreshed automatically by the client SDK while the app remains open.
- An explicit client-side inactivity timer signs the user out after 15 minutes without mouse, keyboard, touch, or scroll activity — necessary because Firebase's own token refresh would otherwise keep a session alive indefinitely regardless of whether anyone is present.

6.4 Data Protection

- Every mutating action — uploads, approvals, skips, batch confirmations, retries, role changes — is written to an append-only audit log with the acting user's identity, never a client-supplied string.
- API-wide rate limiting, Helmet security headers, and an explicit CORS policy are applied to every request.
- Startup configuration validation prevents the server from silently running with missing or malformed credentials.

7. Development Methodology

PaySync was developed using an iterative, vertical-slice methodology rather than a strict waterfall sequence: each phase below produced a working, independently verifiable increment of the real system — a live database schema actually migrated and queried, a live API server actually booted and exercised over HTTP, a live frontend actually type-checked and built — rather than a design document alone. This was a deliberate choice for a system handling real money: correctness at each layer is proven before the next layer is built on top of it, rather than assumed and discovered wrong at integration time.

Each phase in Section 8 is recorded with the same structure: its objective, the concrete modules delivered inside it, the technologies involved, and the exit criteria that mark it complete. Nothing below is described as a vague “future enhancement” — every remaining piece of work has been assigned to a specific, scoped phase.

8. Phased Development Plan

Phase 1 — Discovery, Requirements & System Design

COMPLETED

Objective: Establish the problem, define the data model, and design the API contract and payment lifecycle before writing implementation code.

MODULES & DELIVERABLES

- Problem and requirements analysis — the recipient-verification gap in existing bank and mobile money bulk-transfer workflows.
- Entity-relationship design for payment_sessions, contacts, audit_log, and (later) users.
- Payment state machine design: pending → locked → processing → paid / failed / skipped.
- REST API contract design across upload, verification, batching, retry, and audit endpoints.
- Initial UI/UX direction and wireframes for the Upload, Verify, and Batch Payment screens.

TECHNOLOGIES INVOLVED

PostgreSQL schema design · REST API design · Figma

EXIT CRITERIA

A complete, reviewed entity model and API contract exist and are used, unchanged in their core shape, by every subsequent phase.

Phase 2 — Database & Core Data Pipeline

COMPLETED

Objective: Stand up the real database and build the shared pipeline that turns an uploaded file or manually typed rows into validated, storable contact records.

MODULES & DELIVERABLES

- PostgreSQL schema migrations 001 and 002 — payment_sessions, contacts, audit_log, and enumerated status types.
- File parsing service — tolerant CSV parsing (Papa Parse) and Excel parsing (SheetJS), with flexible column-header matching.
- Shared validation pipeline (uploadService) — one core function used identically by file upload and manual entry, so a fix or rule change applies to both entry points at once.
- Uganda phone number normalization to canonical MSISDN format, and automatic MTN / Airtel network detection from number prefixes.
- Duplicate-contact detection within a single submission.
- Row-count ceiling (5,000 contacts per session) enforced in code, not just advertised in the UI.
- Bulk, chunked database inserts (500 rows per statement) for upload performance at scale.

TECHNOLOGIES INVOLVED

Node.js · Express · PostgreSQL · Multer · Papa Parse · SheetJS

EXIT CRITERIA

A CSV or Excel file of contacts can be uploaded, parsed, validated, deduplicated, and persisted to the database with a complete accept/reject report — verified against a live database, not a mock.

Phase 3 — Payment Engine & Adapter Architecture

COMPLETED

Objective: Build the mechanisms that make bulk payment both safe (no duplicate or premature payment) and network-agnostic (MTN and Airtel behind one interface).

MODULES & DELIVERABLES

- PaymentAdapter interface and PaymentAdapterFactory — the abstraction that lets batch logic call adapter.sendPayment() without ever branching on which network a contact belongs to.
- MtnAdapter — MTN MoMo Disbursements integration: token acquisition, registered-name lookup, payment submission, and status polling, built against MTN's actual three-part sandbox credential model (subscription key, self-provisioned API user, self-provisioned API key).
- AirtelAdapter — Airtel Money Open API integration on the same interface, using OAuth2 client-credentials token acquisition.
- Name resolution service — concurrent, worker-pool-limited verification calls (never firing hundreds of simultaneous requests at once), with exact-match comparison and no fuzzy-matching shortcuts.
- Batch selection engine — SELECT ... FOR UPDATE SKIP LOCKED atomic locking, so concurrent operators can never be assigned the same contact, and a hard requirement that names_match = TRUE before any contact is selectable.
- Batch confirmation and payment execution — the database transaction that locks a batch is committed before any external network call is made, so a slow payment-network response never holds a database lock open.
- Contact review service — explicit approve (after human review of a mismatch) and skip actions, each fully audited, as the only paths that can move a mismatched or unresolved contact into the payable pool.
- Retry and reconciliation service — safe, bounded retries (maximum three attempts) that only ever touch contacts in a definite failed state, never a contact whose true status is still unknown, to eliminate any risk of double payment.
- Session lifecycle logic — automatic transition to completed once every contact reaches a terminal state, and an explicit, audited cancellation path.
- Scheduled reconciliation job (node-cron, every two minutes) that checks the true status of any payment still marked processing, without requiring an operator to remember to do so manually.

TECHNOLOGIES INVOLVED

Node.js · PostgreSQL row-level locking · Axios · node-cron · MTN MoMo API · Airtel Money API

EXIT CRITERIA

The full upload → verify → batch → pay → reconcile pipeline runs end-to-end against a live database and a live Express server; the safety property that an unverified contact can never enter a batch was explicitly tested and confirmed.

Phase 4 — Authentication, Authorization & Security Hardening

COMPLETED

Objective: Replace ad-hoc identity handling with a production-grade identity provider, add role-based access control, and harden the API against common abuse.

MODULES & DELIVERABLES

- Firebase Authentication integration — client SDK for sign-up and sign-in, firebase-admin SDK for server-side ID token verification on every request.

- Auto-provisioning of local application profiles on first authenticated request, including an email-collision fallback that links rather than fails if a profile already exists under that email.
- Role-based access control — cashier and admin roles, with admin-only endpoints for listing users and changing roles.
- Full audit-log integration — every authenticated action is attributed to a real, verified user identity, never a client-supplied string.
- API-wide rate limiting, Helmet security headers, explicit CORS policy, and structured request logging (Morgan).
- Startup environment validation that reports missing configuration clearly rather than failing silently later.
- Database-aware health check endpoint distinguishing “server is up” from “server and database are both reachable.”

TECHNOLOGIES INVOLVED

Firebase Authentication · firebase-admin · Helmet · CORS · express-rate-limit · Morgan

EXIT CRITERIA

A user can sign up, sign in, and reach role-appropriate endpoints; an unauthenticated or improperly-scoped request is rejected with a clear, correct error; all of the above verified live, including a real 11-attempt rate-limit test.

Phase 5 — Frontend Application Development

COMPLETED

Objective: Build the complete client application: every screen an operator or administrator needs, in a distinct, premium visual identity, fully wired to the live API.

MODULES & DELIVERABLES

- React + TypeScript + Vite project scaffold, with a typed API client whose request/response shapes mirror the backend exactly.
- Premium dark “vault” design system — deep bottle-green glass-morphic surfaces, a brass accent reserved for primary actions, and a hand-stamped verification seal as the signature visual mark, built as reusable Tailwind CSS design tokens.
- Core component library — Button, TextField, StampSeal, AppHeader, ProtectedRoute.
- Login and Signup screens, wired to Firebase Authentication.
- Dashboard — real session history pulled from the live API, with entry points into both upload paths.
- Upload screen — drag-and-drop spreadsheet upload with a full accept/reject summary.
- Manual Capture screen — the typed-entry alternative to spreadsheet upload, submitting to the identical backend validation pipeline.
- Verify screen — live name-resolution results with inline approve / skip actions on any mismatch.
- Batch Payment Run screen — the confirmed-batch payment workflow, including the mandatory confirmation prompt before each batch.
- Idle-timeout auto-logout (15 minutes of inactivity), implemented as a reusable hook.

TECHNOLOGIES INVOLVED

React · TypeScript · Vite · Tailwind CSS v4 · React Router · Firebase JS SDK

EXIT CRITERIA

The full application type-checks with zero errors, builds successfully for production, and every screen was verified against the live backend over the Vite development proxy — including a real login round-trip returning a real signed token.

Phase 6 — Live Payment Gateway Integration & Sandbox Verification

IN PROGRESS

Objective: Move from a fully-built but uncredentialed payment layer to one verified against MTN's and Airtel's real sandbox environments, and close the one remaining gap in disbursement coverage.

MODULES & DELIVERABLES

- MTN MoMo Developer Portal registration and subscription to the Disbursements product (self-service, no business documentation required at sandbox level).
- Automated sandbox credential provisioning — a purpose-built script (`provision_mtn_sandbox_user.js`) that performs MTN's two-step self-provisioning process (API user registration, then API key generation) from a single command, rather than the manual Postman workflow MTN's own documentation describes.
- Airtel Africa Developer Portal registration and Application registration to obtain a client ID / client secret pair.
- Live sandbox verification of both name-lookup and disbursement calls against real MTN and Airtel test environments, replacing the current network-egress-blocked test results with genuine responses.
- Airtel disbursement PIN encryption — implementing RSA encryption of the per-transaction PIN using Airtel's merchant-specific public key, the one piece of the Airtel adapter deliberately left unimplemented until real merchant credentials are available, rather than shipped with placeholder or guessed behavior.
- Webhook/callback endpoints (`POST /webhooks/mtn`, `POST /webhooks/airtel`) as a complement to the existing polling-based reconciliation job, allowing near-real-time payment status updates where the sandbox and production callback infrastructure support it reliably.

TECHNOLOGIES INVOLVED

MTN MoMo Developer Portal · Airtel Africa Developer Portal · RSA encryption (Node crypto) · Express webhook routes

EXIT CRITERIA

A real payment can be sent to a real sandbox test number on both MTN and Airtel, its status correctly reconciled, and the Airtel adapter no longer refuses to run for lack of PIN encryption.

Phase 7 — Quality Assurance & Testing

PLANNED

Objective: Establish an automated safety net around the payment-critical logic, and validate the system with its actual intended users before go-live.

MODULES & DELIVERABLES

- Unit test suite (Vitest) for every services-layer module: upload validation and the 5,000-row cap, duplicate detection, phone normalization and network detection, batch selection under simulated concurrency, retry boundary conditions at the maximum-retry limit, and session auto-completion logic.
- Integration test suite exercising real Express routes against a disposable test PostgreSQL database, covering the full upload → verify → batch → pay → reconcile sequence.
- Frontend component tests (Vitest + React Testing Library) for the Capture form, Verify screen approve/skip actions, and Batch Payment confirmation flow.

- End-to-end browser tests (Playwright) driving the complete operator journey from sign-up through a confirmed batch payment.
- Structured User Acceptance Testing with Asinga Technologies cashiers, using small-value real sandbox transactions and a written feedback checklist.
- Security review: automated dependency vulnerability scanning (npm audit) on every build, plus a manual checklist covering authentication bypass attempts, SQL injection attempts against every parameterized query, and rate-limit bypass attempts.
- Load testing the upload and batch-selection pipeline at the advertised 5,000-contact ceiling to confirm response times remain acceptable.

TECHNOLOGIES INVOLVED

Vitest · React Testing Library · Playwright · npm audit

EXIT CRITERIA

A documented, passing automated test suite exists for every payment-critical code path, and at least one full UAT cycle with real cashiers has been completed and its findings addressed.

Phase 8 — Deployment & Infrastructure

PLANNED

Objective: Move the system from a local development sandbox to a real, reachable, production-grade deployment.

MODULES & DELIVERABLES

- Selection and provisioning of an application hosting platform for the Express API server.
- Managed PostgreSQL provisioning with automated, tested daily backups and point-in-time recovery.
- Static hosting and CDN distribution for the built frontend bundle.
- Domain registration and TLS/SSL certificate provisioning for both the frontend and API.
- CI/CD pipeline — automated type-check, build, and test on every change, with deployment gated on all three passing.
- Cutover from the development Firebase project to a dedicated production Firebase project, and from MTN/Airtel sandbox credentials to production credentials.
- Secrets management — every credential (database, Firebase, MTN, Airtel) injected via the hosting platform's environment configuration, never committed to source control.
- Written database backup and disaster-recovery runbook, tested at least once before go-live.

TECHNOLOGIES INVOLVED

CI/CD (e.g. GitHub Actions) · Managed PostgreSQL hosting · Node.js application hosting · CDN / static hosting · TLS/SSL

EXIT CRITERIA

PaySync is reachable at a real, secured domain, backed by production credentials throughout, with a tested backup and recovery procedure in place.

Phase 9 — Monitoring, Support & Maintenance

PLANNED

Objective: Ensure problems are surfaced automatically after go-live, rather than discovered by an operator or, worse, a recipient.

MODULES & DELIVERABLES

- Centralized error tracking wired into both the frontend and backend, capturing stack traces with enough context to reproduce an issue without needing to ask the reporting user for detail.
- Uptime monitoring and alerting on the database-aware health check endpoint.
- Monitoring and alerting on the scheduled reconciliation job's own execution — a silent cron failure should never be the reason a stuck payment goes unnoticed.
- Operational refinements to the admin tooling: session search and filtering, bulk role management for onboarding multiple cashiers at once.
- A written support runbook covering the two situations operators will actually hit: a payment stuck in processing, and a contact that has exhausted its retry limit and needs manual review.

TECHNOLOGIES INVOLVED

Error tracking (e.g. Sentry) · Uptime monitoring · node-cron job observability

EXIT CRITERIA

An on-call operator or administrator is notified automatically within minutes of a payment-critical failure, without needing to be actively watching the system.

Phase 10 — Post-Launch Enhancement Roadmap**PLANNED**

Objective: Extend the platform's reach once the core disbursement workflow has proven itself in production.

MODULES & DELIVERABLES

- Additional payment network support — extending the existing PaymentAdapter interface to further African mobile money providers without modifying the batch engine itself.
- A reporting and analytics dashboard summarizing payment volume, success and failure rates, and per-network breakdowns across historical sessions.
- Export of completed payment runs to common accounting and reconciliation formats for finance-team consumption.
- Multi-currency support beyond Ugandan Shillings, for organizations disbursing across multiple East African markets.
- A native mobile companion app for on-the-go batch review and confirmation, built against the same REST API.

TECHNOLOGIES INVOLVED

Existing adapter interface (extended) · Data visualization tooling · Mobile framework (to be selected)

EXIT CRITERIA

Each enhancement in this phase ships as an independent, scoped addition to the existing architecture rather than a rebuild — the adapter pattern, audit log, and role system established in Phases 3–4 are designed to absorb exactly this kind of extension.

9. Current Project Status Summary

A single-glance snapshot of every phase in this document, for quick reference.

#	Phase	Status
1	Discovery, Requirements & System Design	COMPLETED
2	Database & Core Data Pipeline	COMPLETED
3	Payment Engine & Adapter Architecture	COMPLETED
4	Authentication, Authorization & Security Hardening	COMPLETED
5	Frontend Application Development	COMPLETED
6	Live Payment Gateway Integration & Sandbox Verification	IN PROGRESS
7	Quality Assurance & Testing	PLANNED
8	Deployment & Infrastructure	PLANNED
9	Monitoring, Support & Maintenance	PLANNED
10	Post-Launch Enhancement Roadmap	PLANNED

Five of ten phases are complete and verified against a live running system. Phase 6 requires external registration steps with MTN and Airtel's developer programs — outside engineering control but fully scoped above — before Phases 7 through 10 can proceed in sequence.